

TÀI LIỆU THIẾT KẾ AI AGENT LẬP KẾ HOẠCH HỌC TẬP

1. Mục đích

Tài liệu trình bày thiết kế tổng thể của AI Agent hỗ trợ lập kế hoạch học tập cá nhân hóa dựa trên Ontology. Nội dung tập trung vào kiến trúc Agent, các capability nghiệp vụ, cấu trúc Agent State và Memory, quy trình Planning–Validation–Re-planning, ba loại phương án học tập và kế hoạch thực nghiệm so sánh.

Mục đích của tài liệu là xác định rõ cách Agent tiếp nhận mục tiêu người học, điều phối các công cụ, sinh và xếp hạng nhiều phương án, kiểm tra ràng buộc, giải thích kết quả và cập nhật kế hoạch theo phản hồi của sinh viên hoặc cố vấn. Thiết kế này cần được thống nhất trước khi triển khai để bảo đảm hệ thống được phát triển thành một AI Agent có khả năng lập kế hoạch và sử dụng công cụ, không chỉ là một chatbot giao tiếp bằng ngôn ngữ tự nhiên.

Hệ thống tuân theo các nguyên tắc sau:

- Ontology và Rule Engine là nguồn quyết định tính hợp lệ của kế hoạch học tập.

Agent Orchestrator quản lý State và điều phối các capability theo từng giai đoạn xử lý.

- Candidate Plan Generator chỉ sinh phương án ứng viên; Constraint Validator thực hiện kiểm tra độc lập trước khi Ranking.

- LLM chỉ hỗ trợ diễn giải mục tiêu, phản hồi và kết quả đã có căn cứ; không được tự tạo học phần, tín chỉ, quan hệ tiên quyết hoặc quy định học vụ.

Chỉ các phương án vượt qua Validation mới được xếp hạng, giải thích và hiển thị cho người dùng.

- Ranking và Learning-to-Rank chỉ thay đổi thứ tự ưu tiên, không được thay đổi Ontology, Rules hoặc hard constraints.

- Mọi phương án phát sinh sau phản hồi hoặc Re-planning phải được Validation lại trước khi hiển thị hoặc xác nhận.

1.1. Đóng góp nghiên cứu dự kiến

Nghiên cứu tập trung vào ba đóng góp chính.

Thứ nhất, Ontology-constrained AI Agent. Hệ thống đề xuất một kiến trúc AI Agent trong đó Ontology và Rule Engine đóng vai trò là nguồn quyết định tính hợp lệ của kế hoạch học tập. Agent không trực tiếp tự suy luận hoặc thay đổi các ràng buộc học vụ mà chỉ điều phối quá trình truy xuất tri thức, sinh phương án, kiểm tra ràng buộc, xếp hạng và lập kế hoạch lại. Cơ chế này nhằm giảm và phát hiện các phương án được tạo ra không vi phạm các ràng buộc về tiên quyết, song hành, chuyên ngành, học kỳ mở, giới hạn tín chỉ và quota nhóm môn.

Thứ hai, Grounded Explanation dựa trên Ontology Evidence. Mỗi giải thích của hệ thống phải truy vết được về một hoặc nhiều evidence cụ thể lấy từ Ontology, kết quả truy vấn SPARQL hoặc Rule Engine. LLM chỉ thực hiện nhiệm vụ diễn đạt evidence thành ngôn ngữ tự nhiên và không được tự tạo lý do ngoài các căn cứ đã được hệ thống cung cấp.

Thứ ba, Human-in-the-loop Preference Learning. Hệ thống khai thác lựa chọn và phản hồi của sinh viên hoặc cố vấn để học cách xếp hạng lại các phương án hợp lệ. Preference Learning chỉ tác động đến thứ tự ưu tiên của các phương án đã vượt qua Validator và không được thay đổi Ontology, Rule Engine hoặc các hard constraints.

2. Sơ đồ kiến trúc AI Agent

KIẾN TRÚC AI AGENT LẬP KẾ HOẠCH HỌC TẬP



2.1. Vai trò của Agent Orchestrator

- Chuẩn hóa mục tiêu học tập.
- Khởi tạo Run Context và Student Context.
- Chọn capability theo trạng thái.
- Điều phối truy vấn tri thức, sinh phương án, Validation và Ranking.
- Dừng nhánh lỗi hoặc thiếu dữ liệu.
- Chuyển kế hoạch hợp lệ tới Explanation và Human-in-the-loop.
- Cập nhật feedback, iteration và kích hoạt Re-planning.
- Ghi Evidence/Trace để tái lập và đánh giá.

Agent Orchestrator không trực tiếp quyết định quy tắc học vụ. Ontology và Rule Engine vẫn là nguồn quyết định tính hợp lệ.

3. Công nghệ và các capability

3.1. Technology Stack

Lớp	Công nghệ	Trạng thái
Agent	LangGraph, Pydantic	Dự kiến MVP
Knowledge	RDF/OWL, Protégé, RDFLib, SPARQL	Đang dùng một phần
Planning	Python, Beam Search, heuristic	Đang dùng
Validation	Python Rule Engine + Ontology	Cần đóng gói capability
Data	PostgreSQL, SQLAlchemy	Kiến trúc đích; hiện dùng JSON/CSV
Backend	Flask	Đang dùng
Testing	Pytest	Đang dùng
Future	LTR, pgvector/RAG, LLM	Mở rộng

3.2. Agent Capabilities

Capability	Trách nhiệm	State output
Student Context	Hồ sơ, lịch sử, GPA, tín chỉ, cảnh báo	Student Context
Ontology	CTĐT và quan hệ học phần	Knowledge Context, Evidence
Eligibility	Môn đủ điều kiện và bị loại	Course Space
Candidate Generator	Sinh tập phương án ứng viên từ Course Space và Planning Goal	Candidate Plans
Validator	Kiểm tra hard constraints	Validation Results

Ranking	Chấm điểm và lựa chọn các phương án Safe/Balanced/Accelerated từ tập phương án hợp lệ	Ranked Plans
Risk	Phân tích tiến độ và tải học	Risk Results
Explanation	Giải thích từ evidence	Explanations
Feedback/Re-planning	Chuẩn hóa phản hồi và lập lại	Feedback, Adjustment Request

Danh sách trên mô tả capability logic, chưa ấn định tên hàm, API hoặc schema triển khai.

3.3. Đặc tả Capability

Mỗi capability trong hệ thống là một đơn vị chức năng độc lập do Agent Orchestrator điều phối. Capability chỉ xử lý nhiệm vụ thuộc phạm vi trách nhiệm của mình và trả kết quả về Orchestrator để cập nhật Agent State và điều khiển luồng xử lý.

Ở mức thiết kế, mỗi capability được mô tả thông qua sáu thành phần: **Input, Output, Precondition, Postcondition, Error handling và Provenance**. Capability không được tự suy đoán dữ liệu học vụ còn thiếu, không được nói hard constraints và không được sử dụng kết quả thuộc knowledge snapshot khác với snapshot hiện hành.

Capability	Input	Output	Precondition	Postcondition	Error handling	Provenance
Student Context	Thông tin sinh viên, học kỳ mục tiêu, yêu cầu lập kế hoạch	Student Context gồm lịch sử học tập, học phần đã hoàn thành/chưa đạt, GPA, tín chỉ tích lũy và trạng thái học vụ	Hồ sơ sinh viên tồn tại và có đủ dữ liệu tối thiểu để xác định trạng thái tại học kỳ mục tiêu	Tạo Student Context nhất quán tại thời điểm lập kế hoạch	Nếu thiếu dữ liệu cần thiết, đánh dấu dữ liệu chưa đầy đủ thay vì tự suy đoán	Phiên bản dữ liệu sinh viên hoặc thời điểm snapshot
Ontology/Knowledge	Student Context, chương	Knowledge Context	Xác định được chương	Các quan hệ và quy định học	Nếu thiếu hoặc	Ontology version, rule

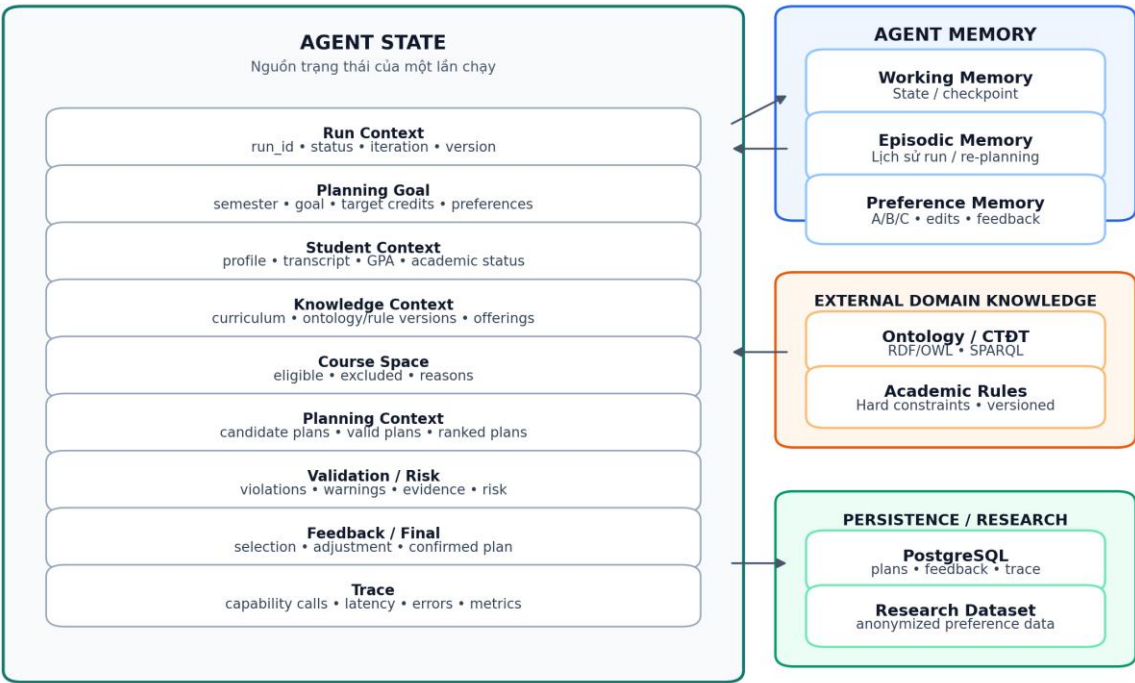
	trình đào tạo, học kỳ mục tiêu, yêu cầu truy xuất tri thức	và evidence từ Ontology , SPARQL hoặc Rule Engine	trình đào tạo và nguồn tri thức tương ứng	vụ sử dụng đúng phiên bản tri thức	không truy xuất được tri thức cần thiết, không đưa ra kết luận học vụ dựa trên suy đoán	version, query identifier, evidence
Eligibility	Student Context, Knowledge Context	Course Space gồm các học phần đủ điều kiện, không đủ điều kiện hoặc chưa thể kết luận	Student Context và Knowledge Context tồn tại và cùng knowledge snapshot	Mỗi học phần được xét có trạng thái eligibility và evidence khi có kết luận	Nếu chưa đủ dữ liệu thì đánh dấu chưa xác định, không tự chuyển thành eligible	Evidence ID, query/rule, knowledge version
Candidate Generation	Course Space, Planning Goal, Adjustment Request nếu Re-planning	Tập Candidate Plans	Course Space đã được xây dựng và Planning Goal hợp lệ	Tạo các candidate plan để chuyển sang Validation; chưa kết luận validity	Nếu không tìm được candidate, trạng thái không tìm được phương án thay vì nói hard	Planning iteration, điều kiện và giới hạn generation

					constrai nts	
Standard Validator	Candida te Plan, Student Context, Knowle dge Context	Validatio n Result gồm valid/inva lid, violations , warnings và evidence	Candidat e đầy đủ dữ liệu và được kiểm tra trên đúng knowled ge snapshot	Mỗi candidate có kết luận validity rõ ràng; chỉ plan valid được chuyển tiếp	Phân biệt plan invalid do vi phạm quy định với lỗi hệ thống hoặc thiếu dữ liệu	Ontology version, rule version, evidence của từng kết luận
Ranking	Các plan valid, Plannin g Goal, Ranking Features	Ranked Plans gồm điểm và thứ tự ưu tiên	Chỉ nhận các plan đã vượt qua Standard Validator	Xếp hạng các plan hợp lệ theo Safe, Balanced hoặc Accelerat ed	Nếu thiếu feature cần thiết thì báo thiếu dữ liệu, không tự suy đoán	Ranking configurati on, feature values, weights/ver sion
Grounded Explanation	Validate d Plan, Ranking Result, Evidenc e	Explanati on có thể truy vết về evidence	Plan đã valid và evidence liên quan tồn tại	Mọi lý do học vụ trong explanati on có căn cứ tương ứng	Không tạo claim học vụ nếu không có evidenc e hỗ trợ	Evidence ID và các nguồn tri thức liên quan
Feedback/Re- planning	Feedbac k của sinh viên/cố vấn và Agent State hiện tại	Adjustme nt Request hoặc vòng lặp kế hoạch mới	Feedback liên quan đến kết quả đã được hệ thống cung cấp	Feedback không sửa trực tiếp plan; candidate mới sau Re- planning	Yêu cầu vi phạm hard constrai nts bị từ chối và phải có	Feedback trace, planning iteration, evide

				phải Validatio n lại	giải thích	
--	--	--	--	----------------------------	---------------	--

Các capability chỉ trao đổi dữ liệu thông qua Agent Orchestrator. Candidate Generator chỉ đề xuất phương án, Standard Validator quyết định tính hợp lệ, Ranking chỉ xếp hạng các phương án hợp lệ, còn Grounded Explanation chỉ diễn đạt các quyết định đã có evidence. Việc phân tách trách nhiệm này giúp bảo đảm tính kiểm soát, khả năng truy vết và tránh để LLM hoặc Ranking can thiệp vào các quy định học vụ.

4. Agent State và Agent Memory



4.1. Mô hình formal State–Action–Tool

Trạng thái của một lần chạy tại bước t được biểu diễn bởi:

Công thức 1: $S_t = \langle r, g, x, k, c_t, v_t, z_t, f_t, e_t, i_t \rangle$

Ký hiệu	Ý nghĩa
r	Run Context: mã lần chạy, trạng thái và thời điểm thực hiện
g	Planning Goal: học kỳ, mục tiêu học tập và số tín chỉ mong muốn
x	Student Context: bản chụp hồ sơ và lịch sử học tập

k	Knowledge Context: bản chụp tri thức và phiên bản dữ liệu
c _t	Tập kế hoạch ứng viên tại bước t
v _t	Kết quả kiểm tra tính hợp lệ
z _t	Danh sách kế hoạch đã được xếp hạng
f _t	Phản hồi của sinh viên hoặc cố vấn
e _t	Evidence và dấu vết thực thi
i _t	Số vòng Planning/Re-planning đã thực hiện

Một snapshot bất biến được xác định bởi:

Công thức 2: KVersion = ⟨studentVersion, curriculumVersion, ontologyVersion, ruleVersion, offeringVersion⟩

Việc lưu đầy đủ các phiên bản giúp tái lập kết quả và phát hiện trường hợp dữ liệu nguồn thay đổi.

Tập hành động điều phối của Agent được định nghĩa như sau:

Công thức 3: A = {LoadContext, QueryKnowledge, BuildCourseSpace, Generate, Validate, Rank, Explain, CollectFeedback, Replan, FinalValidate, Stop}

Mỗi tool hoặc capability T_j được biểu diễn bởi:

Công thức 4: T_j: I_j → O_j ∪ Error_j

Trong đó I_j là dữ liệu đầu vào, O_j là dữ liệu đầu ra hợp lệ và Error_j là tập lỗi có thể xảy ra. Mỗi tool phải công bố input schema, output schema, precondition, postcondition, timeout và provenance.

Hàm chuyển trạng thái của Agent được biểu diễn như sau:

Công thức 5: S_{t+1} = δ(S_t, a_t, O_j)

Trong đó a_t là hành động tại bước t và O_j là kết quả do tool trả về. Chỉ Agent Orchestrator được phép áp dụng hàm chuyển trạng thái δ; các tool riêng lẻ không được tự ý sửa toàn bộ Agent State.

Các bất biến an toàn:

1. Rank và Explain chỉ được thực hiện khi Validator đã kết luận kế hoạch hợp lệ trên đúng phiên bản knowledge snapshot.
2. Confirm chỉ được thực hiện khi Final Validation trên phiên bản dữ liệu hiện hành trả về kết quả hợp lệ.

3. Khi candidate plan hoặc snapshot thay đổi, các kết quả Validation, Ranking và Explanation phụ thuộc phải bị vô hiệu hóa và thực hiện lại.
4. Feedback chỉ cập nhật preference hoặc adjustment request; không được thay đổi knowledge snapshot, Ontology, Rule Engine hoặc nhãn hard constraint.
5. Ranking và Learning-to-Rank chỉ thay đổi thứ tự của các kế hoạch hợp lệ, không được ghi đè kết luận của Validator.

4.2. Cấu trúc Agent State

Nhóm	Nội dung
Run Context	Run ID, status, iteration, giới hạn vòng, thời điểm
Planning Goal	Học kỳ, mục tiêu, tín chỉ, môn ưu tiên/tránh
Student Context	Hồ sơ, lịch sử, GPA, tín chỉ, cảnh báo
Knowledge Context	Phiên bản ontology/rule, CTĐT, môn mở, quan hệ
Course Space	Môn đủ điều kiện, bị loại và lý do
Candidate Plans	Tổ hợp ứng viên và loại phương án
Validation Results	Validity, violation, warning, evidence
Ranked Plans	Top-3, tổng điểm và điểm thành phần
Feedback	Lựa chọn, thứ hạng, điều chỉnh, xác nhận
Evidence / Trace	Căn cứ, lịch sử capability, thời gian, lỗi
Final Result	Plan chọn và kiểm tra cuối

4.3. Cơ chế cập nhật State

Khởi tạo từ yêu cầu hợp lệ và snapshot.

Capability chỉ cập nhật phần thuộc trách nhiệm.

Câu chữ phải được chuẩn hóa trước khi cập nhật.

Mỗi vòng giữ iteration/version và trace cũ.

Candidate Plans có trước Validation Results.

Chỉ plan hợp lệ được đưa vào Ranked Plans.

Feedback tạo Adjustment Request, không sửa trực tiếp plan.

Nguồn dữ liệu thay đổi làm mất hiệu lực kết quả phụ thuộc.

Final Result chỉ được tạo sau Final Validation.

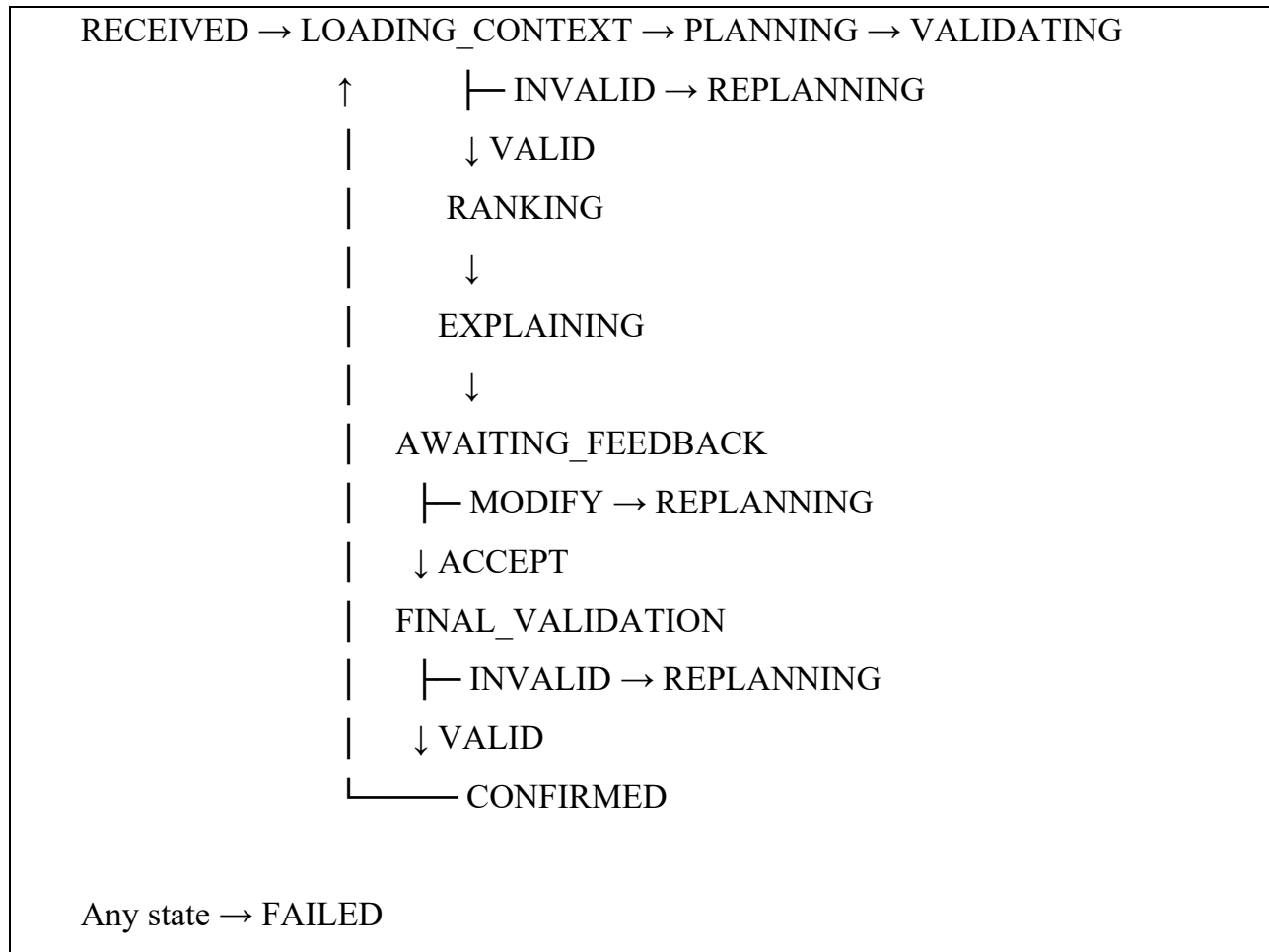
4.4. Agent Memory

Thành phần	Nội dung
Working Memory	Request, snapshots, candidates, Validation, Ranking và feedback của một lần chạy

Episodic Memory	Lịch sử lập kế hoạch, phương án đã xem, vòng chỉnh sửa và quyết định cuối
Preference Memory	Lựa chọn A/B/C, thứ hạng, môn thêm/loại/thay và lý do cố vấn
External Domain Knowledge: Ontology + Rules	CTĐT, quan hệ học phần và Rules đã được phê duyệt

External Domain Knowledge là nguồn tri thức bên ngoài, không phải ký ức do Agent học được. Preference Memory phải được ẩn danh và không được dùng để sửa Ontology hoặc hard constraints.

4.5. Vòng đời Agent State



5. Quy trình Planning–Validation–Re-planning

5.1. Planning

Kiểm tra Planning Goal và Student Context.

Tải Knowledge Context đúng CTĐT/học kỳ.

Xác định môn đủ điều kiện và môn bị loại kèm evidence.

Sinh tập Candidate Plans từ Course Space và Planning Goal.

Loại các candidate trùng lặp trước khi chuyển sang Validation.

Chuyển candidate plans tới Validator.

Planning không tự xác nhận phương án hợp lệ.

5.2. Validation

Validator kiểm tra:

- Môn tồn tại và thuộc đúng CTĐT/chuyên ngành.
- Môn được mở trong học kỳ mục tiêu.
- Tiên quyết, học trước và song hành.
- Giới hạn tín chỉ và cảnh báo học vụ.
- Môn đã hoàn thành, học lại/cải thiện.
- Quota và nhóm môn tự chọn.

Kết quả gồm valid/invalid, violations, warnings, evidence và phiên bản nguồn. Validator chạy trước Ranking, sau Re-planning và ngay trước xác nhận.

5.2.1. Standard Validator

Standard Validator là thành phần độc lập chịu trách nhiệm quyết định tính hợp lệ của một kế hoạch học tập dựa trên Student Context, Ontology, Rule Engine và các dữ liệu học vụ tương ứng.

Validator được thiết kế theo nguyên tắc **deterministic**. Với cùng một Candidate Plan, cùng Student Context và cùng Knowledge Snapshot, Validator phải đưa ra cùng một kết luận về tính hợp lệ.

Có thể biểu diễn:

$$Validation(P, X, K) \rightarrow V$$

trong đó:

- P : Candidate Plan;
- X : Student Context;
- K : Knowledge Snapshot;
- V : Validation Result.

Knowledge Snapshot bao gồm các phiên bản cần thiết như curriculum, ontology, rules và semester offering. Việc lưu version là cần thiết để bảo đảm kết quả có thể được tái lập khi dữ liệu nguồn thay đổi. Tài liệu hiện tại cũng đã xác định KVersion gồm studentVersion, curriculumVersion, ontologyVersion, ruleVersion và offeringVersion.

Standard Validator tuân theo các nguyên tắc sau:

- Không phụ thuộc LLM.

LLM không tham gia vào quá trình quyết định một plan valid hay invalid.

- Validator quyết định hard constraints.

Các điều kiện như prerequisite, co-requisite, semester offering, credit limit, elective quota hoặc quy định học vụ khác được xác định dựa trên Ontology/Rules và dữ liệu nguồn.

- Ranking không được ghi đè Validation.

Một kế hoạch có Ranking Score cao vẫn không được sử dụng nếu Validator xác định kế hoạch đó invalid.

- LLM không được thay đổi kết luận của Validator.

LLM chỉ có thể diễn đạt lại kết quả và evidence.

- Re-planning luôn kéo theo Validation mới.

Mọi candidate được sinh hoặc sửa sau feedback đều phải được kiểm tra lại từ đầu đối với các constraint liên quan.

- Knowledge Snapshot thay đổi làm mất hiệu lực kết quả cũ.

Khi ontology, rule, semester offering hoặc dữ liệu học tập thay đổi, các Validation Result phụ thuộc vào snapshot trước đó không được tiếp tục sử dụng.

- Final Validation trước Confirm.

Trước khi kế hoạch cuối cùng được xác nhận, hệ thống phải kiểm tra lại kế hoạch trên knowledge snapshot hiện hành.

Các nguyên tắc này cũng nhất quán với các invariant đã có trong tài liệu: Rank/Explain chỉ chạy sau Validation, Confirm chỉ sau Final Validation, và khi candidate hoặc snapshot thay đổi thì Validation/Ranking/Explanation phải thực hiện lại.

Vai trò của Standard Validator trong thực nghiệm

Standard Validator đồng thời được sử dụng như một cơ chế đánh giá chung cho các baseline.

Mọi baseline sau khi sinh phương án đều được đưa qua cùng Standard Validator để đo valid-plan rate và các loại violation.

Đối với baseline không sử dụng Ontology trong quá trình generation, Standard Validator không được cung cấp feedback để hỗ trợ việc sinh candidate. Validator chỉ được sử dụng sau khi baseline hoàn thành generation nhằm đánh giá output theo cùng một tiêu chuẩn học vụ.

5.3. Ranking

Ranking chỉ được thực hiện trên các phương án đã vượt qua Standard Validator. Ranking không quyết định tính hợp lệ mà chỉ xác định phương án hợp lệ nào phù hợp hơn với mục tiêu của người học.

Với một phương án hợp lệ P , hệ thống xây dựng vector đặc trưng:

$$F(P) = [f_{goal}, f_{mandatory}, f_{unlock}, f_{credit}, f_{balance}, f_{risk}]$$

Trong đó mỗi feature được chuẩn hóa về khoảng:

$$0 \leq f_j(P) \leq 1$$

Giá trị càng lớn biểu thị phương án càng phù hợp đối với tiêu chí đang xét.

5.3.1. Ranking Features

Với mỗi phương án học tập P đã vượt qua Standard Validator, hệ thống xây dựng vector đặc trưng:

$$F(P) = [f_{goal}(P), f_{mandatory}(P), f_{unlock}(P), f_{credit}(P), f_{workload}(P), f_{safety}(P)]$$

Trong đó mỗi feature được chuẩn hóa về khoảng:

$$f_i(P) \in [0, 1]$$

Giá trị càng lớn thể hiện phương án càng phù hợp đối với tiêu chí tương ứng.

Ranking chỉ được thực hiện trên các phương án đã được Standard Validator xác nhận là hợp lệ. Feature không được sử dụng để thay đổi, bỏ qua hoặc ghi đè bất kỳ kết luận nào của Validator.

Các feature được định nghĩa như sau.

Goal Fit

Goal Fit biểu diễn mức độ phương án đáp ứng Planning Goal và các preference mềm của sinh viên.

Planning Goal được phân rã thành tập các mục tiêu hoặc preference có thể kiểm tra:

$$G = \{g_1, g_2, \dots, g_m\}$$

Ví dụ:

- ưu tiên một học phần;
- tránh một học phần;
- hướng đến hoàn thành đúng tiến độ;
- ưu tiên học vượt;
- ưu tiên một nhóm học phần;
- đạt số tín chỉ mong muốn.

Với mỗi mục tiêu g_j , định nghĩa:

$$s_j(P) \in [0, 1]$$

là mức độ phương án P đáp ứng mục tiêu tương ứng.

Goal Fit được tính:

$$f_{goal}(P) = \frac{\sum_{j=1}^m \alpha_j s_j(P)}{\sum_{j=1}^m \alpha_j}$$

trong đó:

- α_j là mức ưu tiên của mục tiêu g_j ;
- $s_j(P) = 1$ nếu mục tiêu được đáp ứng hoàn toàn;
- $s_j(P) = 0$ nếu mục tiêu không được đáp ứng;
- có thể sử dụng giá trị trung gian trong $[0, 1]$ đối với mục tiêu có mức độ đáp ứng liên tục.

Các mục tiêu người dùng vi phạm hard constraints không được đưa vào Ranking. Trong trường hợp đó, Agent phải thông báo rằng yêu cầu không thể được thực hiện thay vì làm giảm yêu cầu của Validator.

Nguồn dữ liệu:

- Planning Goal;
- Student Context;
- Feedback/Adjustment Request.

Mandatory Priority

Mandatory Priority biểu diễn mức độ phương án ưu tiên các học phần bắt buộc mà sinh viên còn phải hoàn thành.

Gọi:

$$M_{remain}$$

là tập học phần bắt buộc sinh viên chưa hoàn thành và:

$$M(P) = C(P) \cap M_{remain}$$

trong đó $C(P)$ là tập học phần của phương án P .

Điểm thô được tính theo tổng số tín chỉ bắt buộc có trong phương án:

$$r_{mandatory}(P) = \sum_{c \in M(P)} credit(c)$$

Để so sánh giữa các candidate plan trong cùng planning request, feature được chuẩn hóa Min–Max:

$$f_{mandatory}(P) = \frac{r_{mandatory}(P) - r_{mandatory}^{min}}{r_{mandatory}^{max} - r_{mandatory}^{min}}$$

trong đó:

$$r_{mandatory}^{min} = \min_{P' \in V} r_{mandatory}(P')$$

$$r_{mandatory}^{max} = \max_{P' \in V} r_{mandatory}(P')$$

và V là tập các phương án đã vượt qua Standard Validator.

Nếu:

$$r_{mandatory}^{max} = r_{mandatory}^{min}$$

thì:

$$f_{mandatory}(P) = 1$$

cho tất cả phương án trong tập V , vì feature này không tạo ra khác biệt giữa các candidate.

Nguồn dữ liệu:

- Curriculum;
- Student Context;
- Candidate Plan.

Unlock Score

Unlock Score biểu diễn khả năng các học phần trong phương án giúp sinh viên đủ điều kiện tiếp cận các học phần tiếp theo của chương trình.

Với phương án P , gọi:

$$U(P)$$

là tập các học phần chưa hoàn thành mà sau khi hoàn thành các học phần trong P , ít nhất một dependency về prerequisite hoặc course dependency được giải quyết.

Tập $U(P)$ phải được xác định từ Ontology/SPARQL và không được suy diễn bởi LLM.

Điểm thô được tính:

$$r_{unlock}(P) = | U(P) |$$

Để tránh tính trùng một học phần được mở khóa bởi nhiều học phần khác nhau, mỗi học phần trong $U(P)$ chỉ được đếm một lần.

Unlock Score được chuẩn hóa:

$$f_{unlock}(P) = \frac{r_{unlock}(P) - r_{unlock}^{min}}{r_{unlock}^{max} - r_{unlock}^{min}}$$

Nếu:

$$r_{unlock}^{max} = r_{unlock}^{min}$$

thì:

$$f_{unlock}(P) = 1$$

cho toàn bộ tập candidate hợp lệ.

Trong MVP, Unlock Score chỉ xét các quan hệ prerequisite/co-requisite được khai báo chính thức trong Ontology. Không sử dụng quan hệ do LLM tự suy đoán.

Nguồn dữ liệu:

- Ontology;
- SPARQL;
- Curriculum;
- Candidate Plan.

Credit Fit

Credit Fit biểu diễn mức độ tổng số tín chỉ của phương án gần với số tín chỉ mục tiêu mà sinh viên mong muốn.

Gọi:

$$Credit(P) = \sum_{c \in C(P)} credit(c)$$

và:

$$Credit_{target}$$

là số tín chỉ mục tiêu trong Planning Goal.

Độ lệch tín chỉ được tính:

$$\Delta_{credit}(P) = |Credit(P) - Credit_{target}|$$

Gọi $Credit_{min}$ và $Credit_{max}$ là giới hạn tín chỉ áp dụng cho sinh viên tại học kỳ mục tiêu.

Hệ số chuẩn hóa được xác định:

$$D_{credit} = \max(Credit_{target} - Credit_{min}, Credit_{max} - Credit_{target})$$

Credit Fit được tính:

$$f_{credit}(P) = \max\left(0, 1 - \frac{\Delta_{credit}(P)}{D_{credit}}\right)$$

Do tất cả phương án được Ranking đã vượt qua Standard Validator, giới hạn tín chỉ vẫn được quyết định bởi Academic Rules. Credit Fit chỉ đo độ phù hợp với target credits và không quyết định tính hợp lệ.

Ví dụ, nếu:

$$Credit_{target} = 18$$

thì phương án 18 tín chỉ có:

$$f_{credit}(P) = 1$$

và các phương án lệch xa 18 tín chỉ sẽ có điểm thấp hơn.

Nguồn dữ liệu:

- Planning Goal;
- Candidate Plan;
- Credit Policy;
- Student Context.

Workload Balance

Workload Balance biểu diễn mức độ tải học của phương án phù hợp với khả năng học tập hiện tại của sinh viên.

Trong MVP, khi chưa có dữ liệu workload thực nghiệm đáng tin cậy ở cấp học phần, hệ thống sử dụng tải tín chỉ làm proxy chính.

Gọi:

$$Credit_{recommended}$$

là tải tín chỉ tham chiếu phù hợp với trạng thái học vụ của sinh viên.

Giá trị này phải được lấy từ policy hoặc cấu hình hệ thống đã được xác định trước, không được LLM tự tạo.

Độ lệch workload:

$$\Delta_{workload}(P) = |Credit(P) - Credit_{recommended}|$$

Gọi:

$$D_{workload} = \max(Credit_{recommended} - Credit_{min}, Credit_{max} - Credit_{recommended}, 1)$$

Workload Balance được tính:

$$f_{workload}(P) = \max\left(0, 1 - \frac{\Delta_{workload}(P)}{D_{workload}}\right)$$

Giá trị càng gần 1 thể hiện tải học càng gần mức tham chiếu phù hợp với trạng thái của sinh viên.

Trong các phiên bản mở rộng, khi có dữ liệu workload đáng tin cậy, feature này có thể tích hợp thêm:

- số giờ học ước tính;
- độ khó học phần;
- tỷ lệ học phần thực hành/lý thuyết;
- workload lịch sử;
- đánh giá của cố vấn.

Tuy nhiên, công thức mở rộng phải được xác định và cố định trước khi sử dụng trong thực nghiệm chính thức.

Nguồn dữ liệu:

- Student Context;
- Academic Policy;
- Candidate Plan.

Risk/Safety

Risk/Safety biểu diễn mức độ an toàn của phương án đối với trạng thái học tập hiện tại của sinh viên.

Feature này được thiết kế theo chiều:

$$f_{safety}(P) = 1 - Risk(P)$$

trong đó:

$$Risk(P) \in [0, 1]$$

Giá trị Safety càng cao thì phương án càng ít rủi ro.

Trong MVP, Risk Score được xác định từ ba thành phần:

$$Risk(P) = \beta_1 R_{load}(P) + \beta_2 R_{retake}(P) + \beta_3 R_{academic}(P)$$

với:

$$\beta_1 + \beta_2 + \beta_3 = 1$$

Cấu hình ban đầu:

$$\beta_1 = 0.4, \beta_2 = 0.3, \beta_3 = 0.3$$

Trong đó:

Credit Load Risk

$$R_{load}(P) = \frac{Credit(P) - Credit_{min}}{Credit_{max} - Credit_{min}}$$

và kết quả được giới hạn trong khoảng $[0, 1]$.

Phương án càng gần mức tín chỉ tối đa thì mức rủi ro tải học càng cao.

Retake Risk

Gọi $N_{retake}(P)$ là số học phần học lại hoặc cải thiện trong phương án:

$$R_{retake}(P) = \frac{N_{retake}(P)}{|C(P)|}$$

Nếu phương án không chứa học phần:

$$R_{retake}(P) = 0$$

Academic Status Risk

Academic Status Risk được lấy từ trạng thái học vụ hiện tại:

$$R_{academic} \in [0, 1]$$

Ví dụ cấu hình MVP:

- trạng thái học tập bình thường: 0;
- có cảnh báo nhẹ: 0.5;
- đang trong trạng thái cảnh báo học vụ: 1.

Các mức cụ thể phải được mapping từ Academic Rules hoặc cấu hình hệ thống được phê duyệt.

Cuối cùng:

$$f_{safety}(P) = 1 - Risk(P)$$

Hard constraints vẫn do Standard Validator kiểm tra. Một phương án vượt qua Validator nhưng có tải học sát giới hạn vẫn có thể nhận Safety Score thấp.

Nguồn dữ liệu:

- Student Context;
- Candidate Plan;
- Academic Rules;
- Validation Result;

- Risk Result.

Nguyên tắc chuẩn hóa Ranking Features

Mọi Ranking Feature phải có giá trị trong khoảng:

$$[0, 1]$$

và tuân theo cùng một chiều:

$$giá\ trị\ cao\ hơn \Rightarrow phương\ án\ phù\ hợp\ hơn$$

Các feature có miền giá trị tự nhiên như Goal Fit, Credit Fit, Workload Balance và Safety được tính trực tiếp trong khoảng $[0, 1]$.

Các feature dựa trên số lượng hoặc điểm thô như Mandatory Priority và Unlock Score sử dụng Min–Max normalization trong tập candidate hợp lệ của cùng một planning request:

$$Norm(x) = \frac{x - x_{min}}{x_{max} - x_{min}}$$

Nếu:

$$x_{max} = x_{min}$$

thì feature được gán bằng:

$$Norm(x) = 1$$

vì feature đó không tạo ra sự khác biệt giữa các candidate trong cùng lần Ranking.

Mọi giá trị feature cùng dữ liệu nguồn phải được lưu trong Ranking Result để phục vụ provenance và tái lập thực nghiệm.

LLM không được trực tiếp sinh, chỉnh sửa hoặc ghi đè giá trị Ranking Feature.

5.3.2. Công thức Ranking

Sau khi chuẩn hóa các Ranking Features, điểm của phương án P đối với strategy được tính bằng tổng có trọng số:

$$Score_s(P) = \sum_{i=1}^6 w_{s,i} f_i(P)$$

hay cụ thể:

$$Score_s(P) = w_{goal}^s f_{goal}(P) + w_{mandatory}^s f_{mandatory}(P) + w_{unlock}^s f_{unlock}(P) \\ + w_{credit}^s f_{credit}(P) + w_{workload}^s f_{workload}(P) + w_{safety}^s f_{safety}(P)$$

Trong đó:

$$0 \leq w_{s,i} \leq 1$$

và:

$$\sum_{i=1}^6 w_{s,i} = 1$$

Ba strategy Safe, Balanced và Accelerated sử dụng cùng tập feature nhưng có các trọng số khác nhau.

Cấu hình trọng số ban đầu cho MVP được xác định như sau:

Feature	Safe	Balanced	Accelerated
Goal Fit	0.15	0.20	0.20
Mandatory Priority	0.20	0.20	0.15
Unlock Score	0.10	0.15	0.30
Credit Fit	0.15	0.20	0.15
Workload Balance	0.15	0.15	0.10
Safety/Risk	0.25	0.10	0.10
Tổng	1.00	1.00	1.00

Safe

Safe ưu tiên các phương án có mức an toàn cao, tải học phù hợp và giải quyết các học phần bắt buộc còn tồn đọng.

Do đó:

$$w_{safety}^{Safe}$$

được đặt cao nhất.

Safe không có nghĩa là giảm số lượng học phần bằng mọi giá. Mọi phương án Safe vẫn phải đáp ứng Planning Goal ở mức hợp lý và vượt qua toàn bộ hard constraints.

Balanced

Balanced phân bổ trọng số tương đối đồng đều giữa:

- Goal Fit;
- Mandatory Priority;
- Credit Fit;
- Workload Balance;
- Unlock Score;
- Safety.

Mục tiêu của Balanced là tạo phương án cân bằng giữa tiến độ chương trình, khối lượng học tập và mức độ an toàn.

Accelerated

Accelerated ưu tiên khả năng mở khóa các học phần tiếp theo và đáp ứng mục tiêu học vượt.

Do đó:

$$w_{unlock}^{Accelerated}$$

được đặt cao nhất.

Tuy nhiên Accelerated vẫn chỉ được chọn từ các phương án đã vượt qua Standard Validator. Strategy này không được phép vượt credit limit, bỏ qua prerequisite hoặc nói bất kỳ hard constraint nào.

Bộ trọng số trên được sử dụng làm cấu hình khởi tạo cho MVP.

Trong giai đoạn validation, các trọng số có thể được hiệu chỉnh dựa trên:

- advisor agreement;
- acceptance rate;
- NDCG@3;

- MRR;
- số vòng Re-planning.

Sau khi chọn cấu hình cuối cùng trên validation set, toàn bộ trọng số phải được cố định trước khi đánh giá trên test set.

Mỗi Ranking Result phải lưu:

- Ranking strategy;
- feature values;
- weight vector;
- Ranking configuration version;
- tổng Ranking Score.

Điều này cho phép tái lập lý do một phương án được xếp trên hoặc dưới phương án khác.

5.3.3. Độ đa dạng của Top-3 phương án

Ranking Score cao không bảo đảm ba phương án được hiển thị cho người dùng có sự khác biệt có ý nghĩa. Do đó, ngoài Ranking Score, hệ thống áp dụng Diversity Constraint khi lựa chọn Top-3.

Với hai phương án P_i và P_j , gọi:

$$C_i = C(P_i)$$

và:

$$C_j = C(P_j)$$

là tập học phần của hai phương án.

Jaccard Similarity được xác định:

$$J(P_i, P_j) = \frac{|C_i \cap C_j|}{|C_i \cup C_j|}$$

Trong đó:

$$0 \leq J(P_i, P_j) \leq 1$$

Nếu hai phương án giống hoàn toàn:

$$J(P_i, P_j) = 1$$

Nếu hai phương án không có học phần chung:

$$J(P_i, P_j) = 0$$

Độ khác biệt được định nghĩa:

$$D(P_i, P_j) = 1 - J(P_i, P_j)$$

Do đó:

$$0 \leq D(P_i, P_j) \leq 1$$

Giá trị càng lớn biểu thị hai phương án càng khác nhau.

Trong MVP, ngưỡng diversity ban đầu được chọn:

$$\tau_{div} = 0.30$$

Hai phương án được xem là đủ khác biệt khi:

$$D(P_i, P_j) \geq 0.30$$

tương đương:

$$J(P_i, P_j) \leq 0.70$$

Như vậy hai phương án không được trùng nhau quá 70% theo Jaccard Similarity của tập học phần.

Một tập:

$$Top3 = \{P_1, P_2, P_3\}$$

được xem là thỏa diversity nếu:

$$D(P_i, P_j) \geq \tau_{div}$$

với mọi:

$$i \neq j$$

hay:

$$\min_{i \neq j} D(P_i, P_j) \geq \tau_{div}$$

Quy trình lựa chọn Top-3 được thực hiện như sau:

1. Chỉ xét các phương án đã vượt qua Standard Validator.
2. Sắp xếp các phương án theo Ranking Score giảm dần.
3. Chọn phương án có Ranking Score cao nhất làm phương án đầu tiên.
4. Duyệt các phương án tiếp theo theo Ranking Score và chỉ thêm phương án nếu thỏa Diversity Constraint với tất cả phương án đã được chọn.
5. Dừng khi đã chọn đủ ba phương án hoặc khi không còn candidate hợp lệ.

Có thể biểu diễn:

$$P_k \text{ được chọn} \Leftrightarrow Valid(P_k) = true$$

và:

$$\forall P_j \in Selected: D(P_k, P_j) \geq \tau_{div}$$

Nếu không tồn tại đủ ba phương án vừa valid vừa thỏa Diversity Constraint, Agent trả về số phương án hiện có.

Agent không được:

- giảm hard constraints;
- biến phương án invalid thành valid;
- tự thêm học phần không có trong Course Space;

- giảm ngưỡng Validation;

chỉ để tạo đủ ba phương án.

Ngưỡng:

$$\tau_{div} = 0.30$$

là giá trị khởi tạo cho MVP, không phải kết quả được lựa chọn sau khi quan sát test set.

Trong giai đoạn validation, có thể khảo sát một tập giá trị cố định, ví dụ:

$$\tau_{div} \in \{0.20, 0.25, 0.30, 0.35, 0.40\}$$

và lựa chọn giá trị phù hợp dựa trên trade-off giữa:

- số planning request tạo được đủ ba phương án;
- mean pairwise diversity;
- Ranking quality;
- advisor acceptance.

Sau khi lựa chọn trên validation set, τ_{div} phải được cố định trước khi chạy test set.

Ngoài Jaccard Diversity của tập học phần, hệ thống có thể lưu thêm các thông tin mô tả sự khác biệt giữa các phương án như:

- tổng số tín chỉ;
- tỷ lệ học phần bắt buộc/tự chọn;
- Unlock Score;
- Workload Balance;
- Safety Score.

Các thông tin này phục vụ Explanation và phân tích thực nghiệm, nhưng trong MVP không thay thế Jaccard Diversity làm điều kiện chính để lựa chọn Top-3.

5.4. Re-planning

Kích hoạt bởi Validation: candidate vi phạm; Agent ghi lỗi, điều chỉnh không gian tìm kiếm, tăng iteration và chạy lại Planning.

Kích hoạt bởi người dùng: yêu cầu Add, Remove, Replace, Change Target Credits hoặc Change Goal.

Yêu cầu sai bị từ chối kèm giải thích. Agent dừng khi có plan hợp lệ, đạt giới hạn vòng hoặc hết không gian tìm kiếm. Agent không được nói luật để tạo đủ ba phương án.

5.5. Ontology Evidence và Grounded Explanation

Để bảo đảm các giải thích của Agent có thể kiểm chứng, mỗi quyết định liên quan đến học phần phải đi kèm evidence lấy từ Ontology, kết quả truy vấn SPARQL hoặc Rule Engine.

Với một học phần c , tập evidence được biểu diễn:

$$E(c) = \{e_1, e_2, \dots, e_k\}$$

Mỗi evidence lưu các thông tin chính như nguồn tri thức, học phần liên quan, quan hệ hoặc rule được sử dụng, kết quả kiểm tra và phiên bản Ontology.

Ví dụ, khi một học phần không đủ điều kiện do thiếu môn tiên quyết, evidence có thể chứa:

```
evidence_id: EV_001
course: SOT320
constraint: prerequisite
subject: SOT320
predicate: tienQuyet
object: SOT315
student_status: SOT315_not_completed
source: ontology
query_id: Q_PREREQ_01
rule_id: PREREQ_RULE_01
ontology_version: v1.3
```

Mỗi explanation phải lưu tham chiếu đến evidence_id, qua đó có thể truy ngược về triple, truy vấn SPARQL hoặc rule đã dẫn đến quyết định của hệ thống.

Quá trình tạo giải thích được thực hiện theo luồng:

Ontology/Rule Engine → *Evidence* → *Explanation* → *Natural Language*

Trong đó, Ontology và Rule Engine xác định căn cứ học vụ; thành phần Explanation chỉ chuyển các evidence đã có thành nội dung dễ hiểu cho người dùng.

Ví dụ, từ evidence cho biết SOT320 yêu cầu SOT315 nhưng sinh viên chưa hoàn thành SOT315, hệ thống có thể giải thích:

“Học phần SOT320 chưa được đề xuất vì sinh viên chưa hoàn thành học phần tiên quyết SOT315.”

LLM, nếu được sử dụng, chỉ có nhiệm vụ diễn đạt lại evidence và không được tự tạo thêm quy định, quan hệ học phần hoặc nguyên nhân không tồn tại trong evidence.

5.5.1. Vai trò của Ontology Evidence

Ontology Evidence là cơ chế bảo đảm các quyết định học vụ của Agent có thể được kiểm chứng và truy vết.

Evidence không chỉ được sử dụng để tạo explanation mà còn đóng vai trò là nguồn provenance cho các quyết định phát sinh trong Eligibility, Validation, Ranking và Re-planning.

Nguyên tắc chính là:

Mỗi quyết định có ảnh hưởng đến việc chọn, loại hoặc cảnh báo một học phần phải có căn cứ có thể truy ngược về nguồn tri thức cụ thể.

Evidence có thể đến từ:

- Ontology triple;
- kết quả SPARQL query;
- Rule Engine;
- dữ liệu học tập của sinh viên;
- curriculum hoặc semester offering được phiên bản hóa.

5.5.2. Mô hình Evidence

Một evidence được biểu diễn tổng quát:

$$E = \langle e, v, i, d, e, n, c, e, l, d, d, e, c, i, s, i, o, n, s, o, u, r, c, e, f, a, c, t, q, u, e, r, y, r, u, l, e, r, e, s, u, l, t, v, e, r, s, i, o, n \rangle$$

Trong đó:

evidenceId: Định danh duy nhất của evidence.

Decision: Quyết định mà evidence hỗ trợ, ví dụ eligible, excluded, prerequisite satisfied hoặc constraint violated.

Source: Nguồn tạo evidence: Ontology, SPARQL, Rule Engine hoặc dữ liệu nguồn khác.

Fact: Triple hoặc dữ kiện được sử dụng.

Query: SPARQL query hoặc query identifier nếu quyết định được tạo từ truy vấn.

Rule: Rule identifier nếu quyết định liên quan đến một quy tắc học vụ.

Result: Kết quả của việc truy vấn hoặc kiểm tra rule.

Version: Phiên bản ontology, rule, curriculum hoặc source liên quan.

5.5.3. Evidence cho quyết định chọn và loại học phần

Evidence phải tồn tại cho cả quyết định tích cực và tiêu cực.

Ví dụ:

Trường hợp học phần bị loại

Nếu SOT320 yêu cầu SOT315 và sinh viên chưa hoàn thành SOT315:

$$SOT320 \xrightarrow{\text{prerequisite}} SOT315$$

Evidence ghi nhận:

- học phần đang xét: SOT320;
- constraint: prerequisite;
- prerequisite: SOT315;
- trạng thái sinh viên: chưa hoàn thành SOT315;
- nguồn: Ontology;
- query/rule liên quan;
- ontology version.

Decision:

$$SOT320 \rightarrow \text{Ineligible}$$

Trường hợp học phần đủ điều kiện

Nếu sinh viên đã hoàn thành tất cả prerequisite của một học phần, evidence cũng phải ghi lại việc constraint đã được thỏa mãn.

Ví dụ:

$$Pre(SOT320) \subseteq CompletedCourses$$

Decision:

$$SOT320 \rightarrow Eligible$$

Như vậy hệ thống không chỉ trả lời được:

“Vì sao môn này bị loại?”

mà còn trả lời được:

“Dựa vào đâu hệ thống xác định môn này đủ điều kiện?”

5.5.4. Evidence cho Validation

Mỗi violation hoặc warning do Standard Validator tạo ra phải có evidence.

Ví dụ:

$$ValidationResult(P) = \{v, alid = false, violations = [v_1, v_2]\}$$

thì mỗi violation:

$$v_i \rightarrow evidence_i$$

Điều này cho phép truy ngược từ kết luận “plan invalid” đến rule hoặc fact cụ thể đã dẫn đến kết luận đó.

5.5.5. Decision Provenance

Mỗi quyết định quan trọng được lưu dưới dạng một chuỗi provenance:

$$Decision \rightarrow Evidence \rightarrow Query/Rule \rightarrow Ontology Triple(s) \rightarrow Version$$

Ví dụ:

Exclude(SOT320)
↓
EV_001
↓
Q_PREREQ_01 + PREREQ_RULE_01
↓
(S, O, T320 prerequisite SOT315)
↓
ontology_version = v1.3

Nhờ đó có thể trả lời các câu hỏi:

- Quyết định này được tạo từ đâu?
- Quan hệ Ontology nào được sử dụng?
- SPARQL query nào đã chạy?
- Rule nào đã được áp dụng?
- Phiên bản Ontology/Rule nào đã được sử dụng?
- Nếu dữ liệu thay đổi, decision nào cần được tính lại?

5.5.6. Evidence cho Re-planning

Re-planning không chỉ lưu phản hồi của người dùng mà còn phải giữ nguyên nhân dẫn đến việc lập kế hoạch lại.

Hai nhóm nguyên nhân chính:

Re-planning do Validation

Ví dụ candidate vi phạm credit limit:

Candidate → Violation → Evidence → Replanning

Agent sử dụng violation/evidence để điều chỉnh không gian tìm kiếm, nhưng không được thay đổi quy định học vụ.

Re-planning do Feedback

Ví dụ sinh viên yêu cầu loại một học phần.

Feedback được chuyển thành Adjustment Request:

Feedback → AdjustmentRequest → NewCandidate → Validation

Candidate mới vẫn phải qua toàn bộ Validation và evidence tương ứng phải được tạo lại.

5.5.7. Grounded Explanation

Grounded Explanation được xây dựng trên các evidence đã tồn tại trong hệ thống.

Luồng:

Plan + Evidence → GroundedExplanation

không phải:

Plan → LLM → Reason

Nếu LLM được sử dụng, nhiệm vụ của LLM chỉ là chuyển evidence có cấu trúc thành ngôn ngữ tự nhiên.

Mỗi claim liên quan đến quy định học vụ phải tham chiếu đến ít nhất một evidence.

Ví dụ:

Evidence

SOT320 prerequisite SOT315

và:

SOT315 ∉ CompletedCourses

Explanation

“SOT320 chưa được đề xuất vì sinh viên chưa hoàn thành học phần tiên quyết SOT315.”

Explanation không được bổ sung các lý do như:

“SOT320 quá khó đối với sinh viên.”

nếu hệ thống không có evidence tương ứng.

Cơ chế này giúp phân biệt hệ thống đề xuất với một LLM Agent thông thường: reasoning về quy định học vụ được kiểm soát bởi Ontology và Rule Engine; LLM chỉ đảm nhiệm việc diễn đạt kết quả đã có căn cứ.

6. Định nghĩa ba loại phương án

Phương án	Định hướng
Safe – An toàn	Tải vừa phải, ưu tiên môn nợ và giảm rủi ro
Balanced – Cân bằng	Bám tiến độ chuẩn, cân đối tải và nhóm môn
Accelerated – Học vượt	Đề xuất học trước tiến độ khi sinh viên đã đáp ứng đầy đủ các điều kiện học vụ

Mục tiêu lập kế hoạch gồm:

- Hoàn thành đúng hạn: ưu tiên môn bắt buộc và tháo gỡ nút thắt.
- Học vượt tiến độ: đề xuất học trước tiến độ khi sinh viên đã đáp ứng đầy đủ các điều kiện học vụ.
- Mục tiêu người học định hướng Ranking nhưng không thay đổi các ràng buộc bắt buộc.
- Nếu không tạo đủ ba phương án hợp lệ, Agent trả các phương án hiện có và giải thích nguyên nhân. Các phương án Top-3 phải đủ khác biệt để người dùng có cơ sở so sánh.
- Mỗi phương án gồm loại/mục tiêu, danh sách và phân loại môn, tổng tín chỉ, điểm, lý do chọn/loại, cảnh báo, trạng thái Validation và evidence.

7. Human-in-the-loop Preference Learning

Sau khi các phương án học tập đã vượt qua bước Validation, hệ thống cho phép sinh viên hoặc cố vấn lựa chọn phương án phù hợp hơn. Các phản hồi này được sử dụng để cải thiện thứ tự xếp hạng của những phương án hợp lệ trong các lần lập kế hoạch sau.

Giả sử hai phương án hợp lệ p_i và p_j được đưa ra để so sánh. Nếu cố vấn ưu tiên p_i hơn p_j , quan hệ ưu tiên được biểu diễn:

$$p_i \succ p_j$$

Từ lựa chọn này, hệ thống tạo một cặp preference:

$$(x, (p_i), x(p_j)y)$$

trong đó $x(p)$ là vector đặc trưng của phương án và:

$$y = \begin{cases} 1, & \text{nếu } p_i \text{ được ưu tiên} \\ 0, & \text{nếu } p_j \text{ được ưu tiên} \end{cases}$$

Các phản hồi được lưu thành Preference Dataset với một số thông tin chính như:

student_id_anonymous
 planning_request_id
 plan_a_features
 plan_b_features
 selected_plan
 advisor_feedback
 timestamp

Dữ liệu này được sử dụng để huấn luyện mô hình Learning-to-Rank nhằm học hàm xếp hạng:

$$Score_{LTR}(p) = f_{\theta}(x(p))$$

Trong đó f_{θ} là mô hình xếp hạng được học từ dữ liệu preference của sinh viên hoặc cố vấn. Tuy nhiên, Preference Learning chỉ được áp dụng sau bước kiểm tra tính hợp lệ. Một phương án chỉ được đưa vào quá trình xếp hạng khi:

$$Valid(p) = 1$$

Do đó, Preference Learning chỉ có nhiệm vụ học **phương án hợp lệ nào được ưu tiên hơn**, không có quyền quyết định một phương án vi phạm ràng buộc có được chấp nhận hay không. Ontology, Rule Engine và các hard constraints vẫn là thành phần quyết định tính hợp lệ của kế hoạch học tập.

Luồng xử lý có thể được mô tả như sau:

Valid Plans → *Ranking* → *User/Advisor Feedback* → *Preference Dataset*
→ *Learning-to-Rank* → *Updated Ranking*

Trong phạm vi thực nghiệm, preference của cố vấn được sử dụng làm tín hiệu giám sát chính cho Learning-to-Rank; phản hồi của sinh viên có thể được lưu để phân tích hoặc mở rộng trong nghiên cứu tiếp theo.

7.1. Phạm vi triển khai Preference Learning

Preference Learning/Learning-to-Rank là thành phần thuộc kiến trúc đích nhưng không phải điều kiện bắt buộc để hoàn thành phiên bản Agent + Ontology ban đầu.

Quá trình triển khai được chia thành hai giai đoạn.

Giai đoạn 1 – Thu thập Preference

Sau khi Agent + Ontology + Validator + Ranking hoạt động ổn định, hệ thống bắt đầu thu thập preference từ sinh viên và cố vấn.

Các dạng feedback có thể bao gồm:

- lựa chọn một phương án trong Top-3;
- xếp hạng các phương án A/B/C;
- yêu cầu thêm học phần;
- yêu cầu loại học phần;
- yêu cầu thay thế học phần;
- điều chỉnh số tín chỉ mục tiêu;
- nhận xét của cố vấn về lý do lựa chọn.

Dữ liệu này được lưu để hình thành Preference Dataset.

Một preference record có thể chứa:

$PreferenceRecord = \langle s, t, u, d, e, n, t, request, plans, choice, edits, feedback \rangle$

Trong đó thông tin định danh sinh viên phải được ẩn danh khi sử dụng cho nghiên cứu.

7.2. Xây dựng Preference Dataset

Giả sử hệ thống cung cấp hai phương án hợp lệ P_A và P_B .

Nếu cố vấn lựa chọn:

$$P_A \succ P_B$$

thì hệ thống tạo một cặp preference:

$$(F, (P_A), F(P_B)y)$$

với:

$$y = \begin{cases} 1, & P_A \succ P_B \\ 0, & P_B \succ P_A \end{cases}$$

Các lựa chọn này được tích lũy thành Preference Dataset.

Dữ liệu chỉ được sử dụng để huấn luyện sau khi đạt số lượng và chất lượng đủ để thực hiện đánh giá có ý nghĩa.

Không sử dụng dữ liệu của cùng một sinh viên đồng thời trong train và test nhằm hạn chế data leakage.

Điều này nhất quán với Experimental Protocol hiện tại vốn đã yêu cầu chia dữ liệu theo student_id.

7.3. Learning-to-Rank

Sau khi có Preference Dataset, hệ thống có thể huấn luyện một mô hình Learning-to-Rank để học hàm:

$$LTR(F(P)) \rightarrow Score_{LTR}(P)$$

LTR chỉ được thực hiện trên các plan đã được Standard Validator xác nhận là valid.

Do đó:

$$Valid(P) = false \Rightarrow P \notin Ranking$$

bất kể mô hình LTR dự đoán điểm preference như thế nào.

LTR không được:

- thay đổi Ontology;

- thay đổi Rule Engine;
- thay đổi eligibility;
- bỏ qua prerequisite;
- bỏ qua credit limit;
- biến plan invalid thành valid.

Vai trò duy nhất của LTR là:

học phương án hợp lệ nào được cố vấn hoặc người dùng ưu tiên hơn.

Đây cũng chính là nguyên tắc đã có trong tài liệu hiện tại. **7.4. Vai trò của Feedback trước khi có LTR**

Việc chưa triển khai LTR không đồng nghĩa feedback chưa được sử dụng.

Trong giai đoạn Agent + Ontology ban đầu, feedback vẫn được dùng để kích hoạt Re-planning.

Ví dụ:

*Top3 → Feedback(Remove course) → AdjustmentRequest → Replanning
→ Validation → Ranking*

Đồng thời feedback được lưu lại để tạo dữ liệu phục vụ Preference Learning trong giai đoạn tiếp theo.

Như vậy Feedback có hai vai trò:

8. Kế hoạch thực nghiệm so sánh

8.1. Các baseline

Mã	Phương pháp
BL-01	Rule-based
BL-02	Greedy
BL-03	Beam Search hiện tại
BL-04	Agent không Ontology, chỉ chạy offline
BL-05	Agent có Ontology
BL-06	Agent có Ontology + Preference Learning/LTR

BL-04 – Agent without Ontology

BL-04 sử dụng cùng:

- Student Context;
- Planning Goal;
- danh sách học phần;
- số tín chỉ;
- lịch sử học;
- cùng giới hạn số candidate;
- cùng giới hạn thời gian/tìm kiếm.

Tuy nhiên BL-04 **không được sử dụng các tri thức quan hệ và ràng buộc từ Ontology**, bao gồm:

- prerequisite;
- co-requisite;
- course–major relation;
- semester offering;
- elective quota;
- ontology-derived eligibility;
- SPARQL evidence.

BL-04 sinh kế hoạch dựa trên dữ liệu dạng phẳng và heuristic/ranking tương tự nhưng không nhận phản hồi từ Ontology Validator trong quá trình generation.

Sau khi sinh xong, tất cả output của BL-04 vẫn được đưa qua **cùng một Standard Validator** như các baseline còn lại.

Nhờ đó ta đo được:

$$Impact_{ontology} = Metric(BL05) - Metric(BL04)$$

và tránh việc BL-04 được đánh giá theo tiêu chuẩn khác BL-05.

8.2. Ablation cho grounded explanation

Mã	Cấu hình
EX-ABL-01	Agent + Ontology, không cung cấp explanation
EX-ABL-02	Agent + Ontology + grounded explanation

Hai cấu hình giữ nguyên candidate plans, Validation và Ranking để đo riêng ảnh hưởng của explanation đến acceptance rate.

8.3. Chỉ số đánh giá

Nhóm	Chỉ số
Validity	Valid plan rate; violations theo loại
Agreement	Top-1 advisor agreement; acceptance
Ranking	NDCG@3; MRR
Editing	Số vòng Re-planning; số môn sửa; edit distance
Diversity	Tỷ lệ đủ ba loại; độ khác biệt Top-3
Explanation	Groundedness, completeness, clarity, usefulness
Performance	Latency p50/p95; capability calls; lỗi; chi phí

8.4. Câu hỏi nghiên cứu

RQ1: Ontology có tăng tỷ lệ plan hợp lệ so với Agent không Ontology?

RQ2: Agent có Ontology có tăng đồng thuận cố vấn so với các baseline?

RQ3: Preference Learning/LTR có cải thiện NDCG@3, MRR và giảm chỉnh sửa?

RQ4: Grounded explanation có cải thiện groundedness, mức độ hữu ích và acceptance rate so với explanation không grounded hoặc không có explanation hay không?

8.5. Experimental Protocol

Dữ liệu được chia thành train, validation và test theo student_id, bảo đảm các planning request của cùng một sinh viên không xuất hiện đồng thời ở nhiều tập dữ liệu nhằm hạn chế data leakage. Tập test, Ontology, Rule Engine, curriculum snapshot và các tham số của phương pháp được đóng băng trước khi thực hiện đánh giá cuối cùng.

Để bảo đảm tính công bằng khi so sánh các baseline, tất cả phương pháp được đánh giá trên cùng một tập dữ liệu đầu vào và cùng điều kiện thực nghiệm. Cụ thể, mỗi baseline sử dụng cùng hồ sơ sinh viên, planning request, curriculum snapshot, tập test, random seed, giới hạn tìm kiếm, số vòng lặp tối đa và số lượng candidate được phép sinh.

Có thể biểu diễn nguyên tắc này như sau:

$$D_{input}^{(i)} = D_{input}$$

với mọi baseline i .

Sau khi mỗi phương pháp sinh ra các phương án học tập, toàn bộ kết quả đều được đưa qua cùng một **Standard Validator** để đánh giá tính hợp lệ:

$$Output_i \rightarrow StandardValidator \rightarrow Evaluation$$

Đối với BL-04, Standard Validator không tham gia vào quá trình sinh phương án nhằm giữ đúng điều kiện “Agent không Ontology”. Tuy nhiên, sau khi BL-04 hoàn tất generation, đầu ra vẫn được kiểm tra bằng cùng Validator chuẩn như các baseline khác. Cách thiết kế này giúp bảo đảm mọi phương pháp được đánh giá trên cùng một bộ tiêu chí học vụ.

Các chỉ số chính được sử dụng để trả lời các câu hỏi nghiên cứu gồm:

RQ1 – Tính hợp lệ của phương án

Tỷ lệ phương án hợp lệ được xác định bởi:

$$ValidPlanRate = \frac{\#ValidPlans}{\#GeneratedPlans}$$

Chỉ số này phản ánh khả năng của từng phương pháp trong việc tạo ra kế hoạch không vi phạm các hard constraints.

RQ2 – Mức độ đồng thuận với cổ vấn

Mức độ phù hợp giữa phương án xếp hạng cao nhất của hệ thống và lựa chọn của cổ vấn được đo bằng:

$$Top1Agreement = \frac{\#Top1\ plans\ được\ cổ\ vấn\ lựa\ chọn}{\#PlanningRequests}$$

Chỉ số này được sử dụng để đánh giá mức độ phù hợp thực tế của phương án mà hệ thống ưu tiên.

RQ3 – Chất lượng Ranking

Đối với các phương án được xếp hạng, sử dụng các chỉ số:

- NDCG@3;
- MRR.

Các chỉ số này phản ánh mức độ phù hợp của thứ tự Ranking so với preference của cổ vấn hoặc sinh viên.

RQ4 – Chất lượng Grounded Explanation

Chất lượng explanation được đánh giá dựa trên khả năng truy vết nội dung giải thích về evidence cụ thể.

Groundedness được xác định bởi:

$$Groundedness = \frac{\#Claims \text{ có evidence}}{\#Claims}$$

Ngoài ra, mức độ bao phủ evidence được xác định bởi:

$$EvidenceCoverage = \frac{\#Decisions \text{ được giải thích bằng evidence}}{\#Decisions \text{ cần giải thích}}$$

Bên cạnh các chỉ số định lượng, cố vấn có thể đánh giá explanation theo các tiêu chí:

- clarity;
- usefulness;
- trust;
- acceptance.

Kết quả thực nghiệm được báo cáo bằng giá trị trung bình, độ lệch chuẩn, khoảng tin cậy và effect size khi phù hợp.